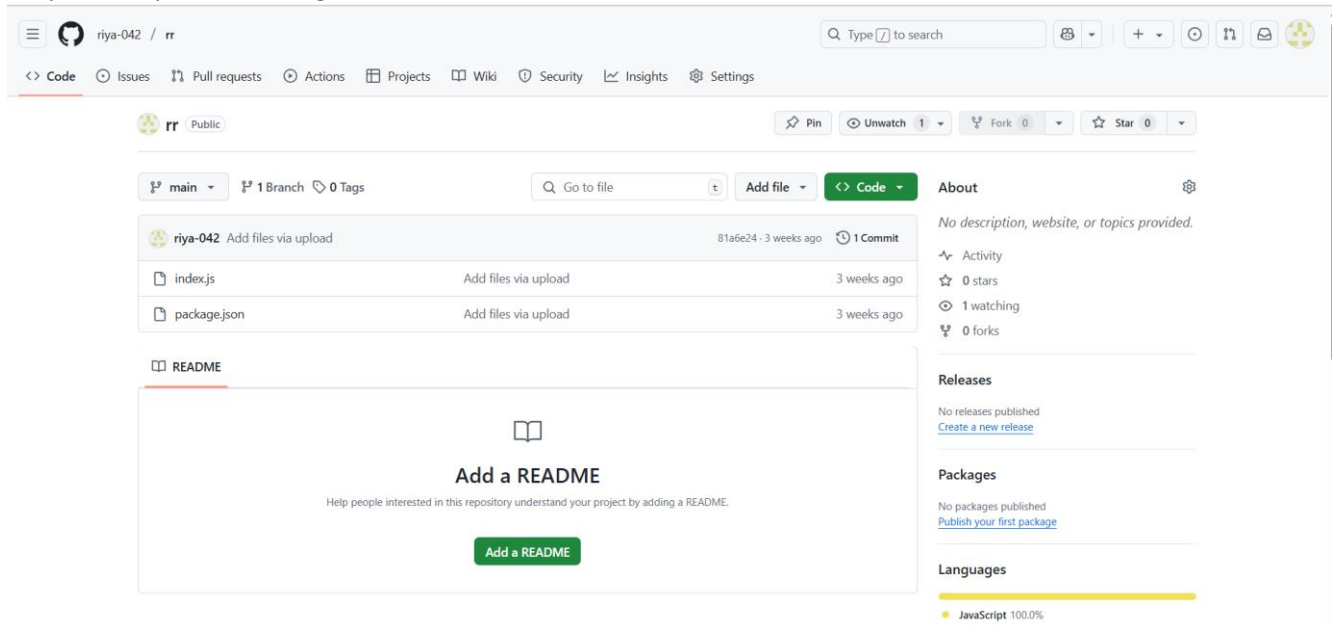


Assignment No:12

Title: Deploy and run the project in AWS without using the port.

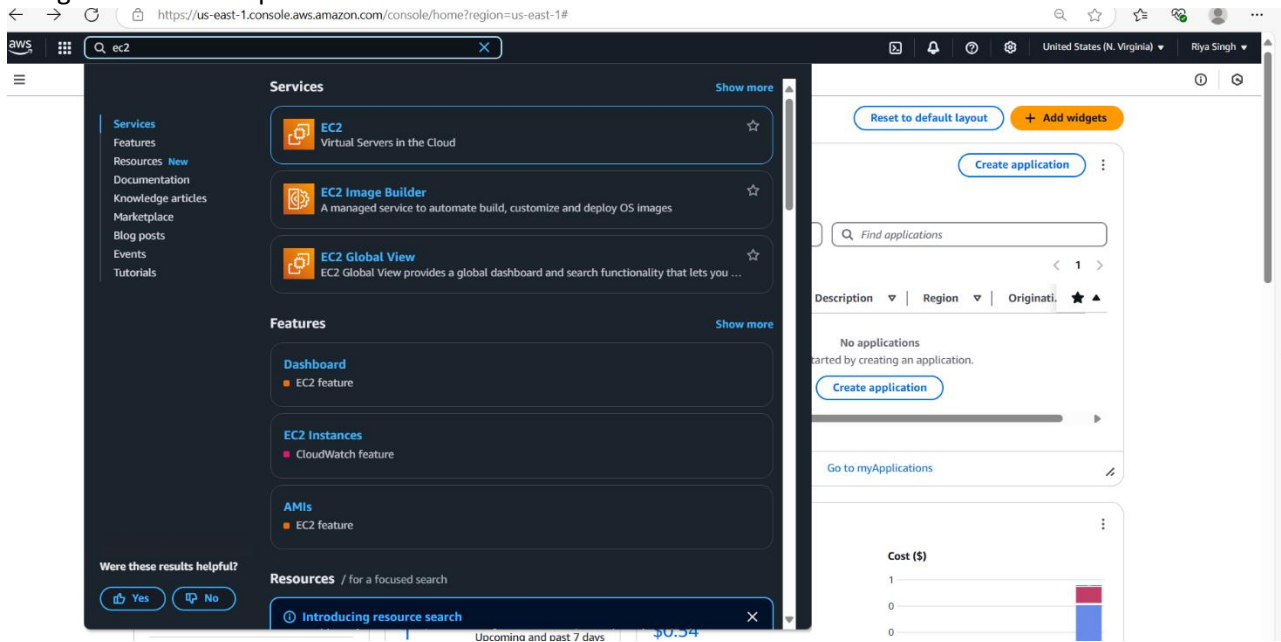
Step-1:

Upload required files to github



Step-2:

Log into AWS and open EC2.



Step-3:

Open the EC2

The screenshot shows the AWS Management Console for the 'United States (N. Virginia)' region, user 'Riya Singh'. The left-hand navigation menu is open, showing the 'EC2' section. The main content area displays the 'Amazon Elastic Compute Cloud (EC2)' dashboard. It features a large heading 'Amazon Elastic Compute Cloud (EC2)' with the subtext 'Create, manage, and monitor virtual servers in the cloud.' Below this, a paragraph describes EC2 as the broadest and deepest compute platform. To the right, there's a 'Launch a virtual server' section with a 'Launch instance' button and a 'View dashboard' button. Further down, the 'Benefits and features' section highlights 'EC2 offers ultimate scalability and control' with a list of features: highest level of control, widest variety of server size options, widest availability of operating systems, and global scalability. A 'Get started' section offers 'Get started walkthroughs' and a 'Get started tutorial' link.

Step-4:

Create ec2 instance and add the below bash commands in user data section.

The screenshot shows the 'Launch an instance' wizard in the AWS Management Console. The browser address bar shows the URL: 'https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:'. The page has a blue header with a message: 'It seems like you may be new to launching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices.' Below this, the 'Launch an instance' section is active. It includes a 'Name and tags' section with a text input field containing 'riyaserver' and an 'Add additional tags' button. The 'Application and OS Images (Amazon Machine Image)' section is also visible, with a search bar and a 'Quick Start' section. The 'Quick Start' section displays a grid of operating system logos: Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. To the right, the 'Summary' section shows the configuration: 'Number of instances: 1', 'Software Image (AMI): Canonical, Ubuntu, 24.04, amd64...', 'Virtual server type (instance type): t2.micro', 'Firewall (security group): New security group', and 'Storage (volumes): 1 volume(s) - 8 GiB'. A 'Free tier' notification is also present at the bottom right, stating that the user gets 750 hours per month of t2.micro instance usage for free.

Applications or agents that use V1 for instance metadata access will break.

Metadata response hop limit [Info](#)
2

Allow tags in metadata [Info](#)
Select

User data - optional [Info](#)
Upload a file with your user data or enter it in the field.
[Choose file](#)

```
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
curl -SL https://deb.nodesource.com/setup_16.x | sudo -E bash -
apt-get install -y nodejs
git clone http://github.com/riya-042/m.git
cd Repo1
npm install
node index.js
```

☐ User data has already been base64 encoded

Summary

Number of instances [Info](#)
1

Software image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-084568db4383264d4

Virtual server type (instance type)
t2.micro

Firewall (security group)
default

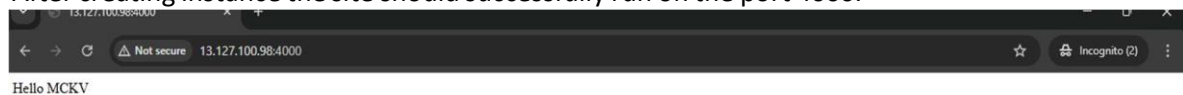
Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet.

[Cancel](#) [Launch instance](#) [Preview code](#)

Step-5:

After creating instance the site should successfully run on the port 4000.



Step-6:

Now log into bitvise and cd ../.. to go to root directory then open etc with cd etc and nginx with cd nginx and sites-available with cd sites-available then change the permissions of the default file using sudo chmod 777 default. Then edit the default file with nano default and change the code to what is given below and close the editor then restart the nginx with sudo systemctl restart nginx.

```
ubuntu@13.201.80.38:22 - Bitwise xterm - ubuntu@ip-172-31-0-238: /etc/nginx/sites-available
ubuntu@ip-172-31-0-238:~$ cd ../../
ubuntu@ip-172-31-0-238:/$ ls
bin          dev          lib          lost+found  opt          run          snap        tmp
bin.usr-is-merged  etc          lib.usr-is-merged  media       proc         sbin         srv          usr
boot         home         lib64        mnt         root         sbin.usr-is-merged  sys          var
ubuntu@ip-172-31-0-238:/$ cd etc
ubuntu@ip-172-31-0-238:/etc$ cd nginx
ubuntu@ip-172-31-0-238:/etc/nginx$ ls
conf.d          koi-utf      modules-available  proxy_params    sites-enabled    win-utf
fastcgi.conf    koi-win      modules-enabled    scgi_params     snippets
fastcgi_params  mime.types   nginx.conf         sites-available uwsgi_params
ubuntu@ip-172-31-0-238:/etc/nginx$ cd sites-available
ubuntu@ip-172-31-0-238:/etc/nginx/sites-available$ ls
default
ubuntu@ip-172-31-0-238:/etc/nginx/sites-available$ nano default
ubuntu@ip-172-31-0-238:/etc/nginx/sites-available$ sudo systemctl restart nginx
ubuntu@ip-172-31-0-238:/etc/nginx/sites-available$
```

```
GNU nano 7.2          default
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://localhost:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

[ Directory '.' is not writable ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

Step-7:

Now the site runs on IP without giving the port.

